

# TỔNG HỢP BÀI TẬP LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG (OOP)

Biên soạn: Kiệt - UET

January 12, 2026

## Contents

|   |                                    |   |
|---|------------------------------------|---|
| 1 | HỆ THỐNG MÔ PHỎNG TRẬN ĐẤU ROBOT   | 1 |
| 2 | SỐ PHỨC (COMPLEX NUMBER)           | 3 |
| 3 | LỚP TAM GIÁC (TRIANGLE)            | 3 |
| 4 | XỬ LÝ THỜI GIAN (TIMESTAMP)        | 4 |
| 5 | PHÂN SỐ (FRACTION)                 | 5 |
| 6 | TÀI KHOẢN NGÂN HÀNG (BANK ACCOUNT) | 5 |

---

## 1. HỆ THỐNG MÔ PHỎNG TRẬN ĐẤU ROBOT

**Mô tả:** Xây dựng hệ thống mô phỏng trận đấu giữa các loại Robot với cơ chế tính toán sát thương riêng biệt (Áp dụng Kế thừa & Đa hình).

### 1. Class Robot (Lớp Cha)

- **Attributes:** name, hp (máu), atk (sức tấn công).
- **Methods:**
  - `__init__(name, hp, atk)`: Khởi tạo các chỉ số.
  - `is_alive()`: Trả về `True` nếu `HP > 0`, ngược lại `False`.
  - `receive_damage(amount)`: Giảm HP đi một lượng `amount` (HP tối thiểu là 0).
  - `attack(other)`: Gây sát thương lên Robot `other`. Mặc định sát thương bằng `atk`.

## 2. Class Tanker (Lớp Con - kế thừa Robot)

- **Đặc điểm:** Có thêm `defense` (giáp - cố định) và `shield` (khiên).
- **Methods:**
  - `__init__(name, hp, atk, defense)`: Khởi tạo như Robot, thêm `defense`. Mặc định `shield = 0`.
  - **Skill "Fortify"**: Mỗi khi Tanker tấn công (`attack`), tự tăng cho bản thân 1 điểm `shield`.
  - **Cơ chế Nhận Sát Thương (Ghi đè `receive_damage`)**:
    - \* Công thức: Sát thương thực tế = `amount - defense - shield`.
    - \* Lưu ý: Sát thương thực tế  $\geq 0$ .
    - \* **Cơ chế vỡ khiên**: Nếu `shield > 0`, sau khi tính toán sát thương xong, `shield` lập tức về 0 (bất kể có chặn hết đòn hay không).

## 3. Class Assassin (Lớp Con - kế thừa Robot)

- **Đặc điểm:** Có thêm `crit_count` (đếm chí mạng) và `stealth` (tàng hình).
- **Methods:**
  - `__init__(name, hp, atk)`: Khởi tạo như Robot. Mặc định `crit_count = 0`, `stealth = False`.
  - **Cơ chế Tấn Công (Ghi đè `attack`)**:
    - \* Tăng `crit_count` lên 1.
    - \* Nếu `crit_count == 3`: Gây **200% sát thương** (`atk × 2`). Sau đó reset `crit_count = 0`.
    - \* Nếu `crit_count < 3`: Gây sát thương bằng `atk` bình thường.
  - **Cơ chế Nhận Sát Thương (Ghi đè `receive_damage`)**:
    - \* Nếu đang `stealth == True`: Sát thương nhận vào bị chia đôi (`amount // 2`).
    - \* Nếu không `stealth`: Nhận sát thương bình thường.
  - **Nội tại ẩn thân**: Ngay sau khi nhận sát thương, nếu  $HP \leq 30\%$  so với HP ban đầu, kích hoạt `stealth = True` vĩnh viễn cho đến chết.

## Dữ liệu (Input/Output)

- **Input:**
  - Dòng 1: TANK [Name] [HP] [ATK] [DEF]
  - Dòng 2: ASN [Name] [HP] [ATK]
  - Dòng 3: Số nguyên  $N$  (số lượt).
  - $N$  dòng tiếp theo: Tên Robot thực hiện hành động tấn công.
- **Output:** In trạng thái cuối cùng (Tanker trước, Assassin sau). Format: [Name] : HP=[CurrentHP]

## 2. SỐ PHỨC (COMPLEX NUMBER)

**Mô tả:** Thiết kế lớp `ComplexNumber` biểu diễn số phức  $z = a + bi$  ( $i^2 = -1$ ).

### Yêu cầu kỹ thuật

Override các phương thức magic methods của Python:

1. `__init__(self, real, imag)`: Lưu phần thực và ảo (float).
2. `__add__(self, other)`: Toán tử  $+$ . Trả về đối tượng mới.
3. `__mul__(self, other)`: Toán tử  $*$ . Trả về đối tượng mới.
4. `__abs__(self)`: Hàm `abs()`. Trả về mô-đun (float).
5. `__str__(self)`: Trả về chuỗi dạng " $a+bi$ " hoặc " $a-bi$ ".
6. `conjugate(self)`: Trả về số phức liên hợp (đối tượng mới).

### Dữ liệu (Input/Output)

**Ràng buộc:** Chương trình chính phải thực hiện:  $z_1 + z_2$ ,  $z_1 \times z_2$ ,  $|z_1 \times z_2|$ , và  $\text{conj}(z_1 \times z_2)$ .

- **Input:**

- Dòng 1:  $a_1 \ b_1$  (Thực và ảo của  $z_1$ ).
- Dòng 2:  $a_2 \ b_2$  (Thực và ảo của  $z_2$ ).

- **Output:** (Mỗi giá trị làm tròn 2 chữ số thập phân)

- Dòng 1: Tổng  $z_1 + z_2$
  - Dòng 2: Tích  $z_1 \times z_2$
  - Dòng 3: Mô-đun của tích  $|z_1 \times z_2|$
  - Dòng 4: Liên hợp của tích  $\text{conj}(z_1 \times z_2)$
- 

## 3. LỚP TAM GIÁC (TRIANGLE)

**Mô tả:** Khai báo lớp `Triangle` để xử lý thông tin tam giác.

### Yêu cầu kỹ thuật

- **Attributes:** `edge1`, `edge2`, `edge3` (kiểu int).

- **Methods:**

- `__init__(self, a, b, c)`: Khởi tạo độ dài 3 cạnh.
- `getArea(self)`: Trả về diện tích (double) nếu tam giác hợp lệ, ngược lại trả về  $-1$ .

## Dữ liệu (Input/Output)

- **Input:** Một dòng chứa 3 số nguyên  $a$   $b$   $c$ .
  - **Output:** Nếu không hợp lệ in "invalid". Nếu hợp lệ in diện tích làm tròn 2 chữ số thập phân.
- 

## 4. XỬ LÝ THỜI GIAN (TIMESTAMP)

**Mô tả:** Xây dựng lớp `TimeStamp` (Giờ-Phút-Giây) để tìm thời điểm trước/sau.

### Yêu cầu kỹ thuật

- `isValid()`: Kiểm tra thời điểm có hợp lệ không.
- `nextSecond()`: Trả về thời điểm kế tiếp sau 1 giây (nếu hợp lệ).
- `previousSecond()`: Trả về thời điểm trước đó 1 giây (nếu hợp lệ).

## Dữ liệu (Input/Output)

- **Input:** Chuỗi định dạng HH:MM:SS.
- **Output:** Nếu sai định dạng in "INVALID". Nếu đúng in 2 dòng: thời điểm kế tiếp và thời điểm trước đó.

## 5. PHÂN SỐ (FRACTION)

Mô tả: Thiết kế lớp `Fraction` để thực hiện 4 phép tính cơ bản.

### Yêu cầu kỹ thuật

1. **Tối giản tự động:** Phân số phải được tối giản ngay khi khởi tạo (chia cho GCD).
2. **Quy ước dấu:** Mẫu số luôn dương ( $den > 0$ ). Dấu âm chuyển lên tử.
3. **Methods:** `add`, `sub`, `mul`, `div`. Tất cả trả về đối tượng `Fraction` mới đã tối giản.
4. **Hiển thị (`__str__`):** Dạng "tử/mẫu" (VD:  $1/2$ ,  $-3/4$ ).

### Dữ liệu (Input/Output)

- **Input:** Một dòng  $a\ b\ op\ c\ d$  (Tử 1, Mẫu 1, Toán tử, Tử 2, Mẫu 2).
  - **Output:** Kết quả phép tính ở dạng tối giản.
- 

## 6. TÀI KHOẢN NGÂN HÀNG (BANK ACCOUNT)

Mô tả: Mô phỏng giao dịch nạp/rút tiền.

### Yêu cầu kỹ thuật

- `__init__(owner, balance)`: Tên chủ (string) và số dư (float).
- `deposit(amount)`: Nạp tiền (yêu cầu  $amount > 0$ ).
- `withdraw(amount)`: Rút tiền (yêu cầu  $amount > 0$ ). Nếu  $amount > balance$ , hủy giao dịch (không trừ tiền, không báo lỗi).
- `__str__()`: Trả về "Owner: Balance" (Balance làm tròn 2 số thập phân).

### Dữ liệu (Input/Output)

- **Input:** Dòng 1 chứa Tên và Số dư đầu. Dòng 2 chứa số  $N$ .  $N$  dòng tiếp theo chứa lệnh (DEPOSIT/WITHDRAW) và số tiền.
- **Output:** Số dư cuối cùng (làm tròn 2 chữ số thập phân).